39. Caching, Redis Patterns, Idempotency

39. Caching, Redis Patterns, Idempotency

Phase: Phase 6 — Systems and infrastructure
Estimated time: 1–2 days
Slug: `39-caching-redis`

Learning objectives

- Apply cache-aside pattern
- Choose TTLs and invalidation strategies
- Implement idempotency keys for writes
- Understand stampede risks

Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

Why this matters

Caching and idempotency protect databases and make retries safe. Cache-aside and Redis patterns appear in every production stack.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

Core concepts

- Cache-aside: read cache → miss → load DB → fill cache.
- Set TTLs; plan invalidation on writes.
- Idempotency keys dedupe retried side effects.
- Stampede protection: singleflight or locks.
- Redis is a tool — model keys carefully with namespaces.

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-caching, && cd /tmp/gw-caching,
go mod init example.com/scratch
```

Then create `main.go`:

```
package main
```

```go
import "fmt"

func CacheAside(mem map[string]string, key string, load func() string) string {
    if v, ok := mem[key]; ok {
        return v
    }
    v := load()
    mem[key] = v
    return v
}

func main() {
    mem := map[string]string{}
    loads := 0
    get := func() string {
        loads++
        return "v"
    }
    fmt.Println(CacheAside(mem, "a", get), CacheAside(mem, "a", get), loads)
}
```

Run it:

```
go run .
```

Expected output:

```
v v 1
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

Mini-lab (`go run` steps)

1. Implement in-memory cache-aside.
2. Add TTL by storing expiry time.
3. Simulate retries with an idempotency set.

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

Curriculum notes (expanded from the compact draft)

Cache-aside

1. Read cache
2. On miss: read DB, populate cache
3. On write: update DB, invalidate key

Redis roles

- Cache

- Rate limiting
- Distributed locks (carefully)
- Queues / streams

Idempotency

For payment-like POSTs, clients send `Idempotency-Key`. Server stores result of first processing and replays it — critical for at-least-once delivery worlds.

Thundering herd

When hot key expires, many requests hit DB. Mitigate with singleflight / locking / soft expiry.

Failure modes to watch while coding

- Caching without TTL
- Using Redis locks without expiry
- Assuming cache is always consistent with DB

When a test fails, ask which failure mode you hit before changing code at random.

Idioms vs Python pitfalls

- functools.cache / Redis clients — same patterns.
- Mutable default dicts vs make(map).
- Think about consistency: cache can be stale.

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would "work". The goal is a Go-shaped intuition.

Going deeper

Document consistency: stale reads vs always-fresh after write.

Practice checklist

Before the exercise, confirm you can:

1. Explain "Caching, Redis Patterns, Idempotency" aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson's rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked

example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

Common mistakes (especially from Python instincts)

- Caching without TTL
- Using Redis locks without expiry
- Assuming cache is always consistent with DB

Exercises

Work in `exercises/39-caching-redis/`. Read that folder's `README.md` before you code.

What you will build

Implement cache-aside or idempotency stub per tests.

Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/39-caching-redis/` until you have an honest failing attempt.

Self-check

Answer without scrolling up:

1. What is cache-aside?
2. Why idempotency keys?
3. What is a stampede?

Further reading

- https://redis.io/docs/manual/patterns/